

CW GIT

Ehsan Esmaeili

2024 nov 18

Basic git commands

0.1 Git add

git add -u

u stand for update

this operator add changed or deleted file to this stage but new file not added to this stage.

advantage of this oprator is you could add changes to your project without adding a new file which is not ready yet.

git add .

this oprator add alls files such as new files , deleted files and changed files but issue of this opreator is you add all file which you not refer to add it

git add -a

this opreator such as git add . but this save all changes in any branch

0.2 Gitignore

role of the gitignore

this opreator soecify file wich shoud be ignore in project

value of gitignore

git usfull for:

- 1 clutter reduction
- 2 sensitive information
- 3 build artifacts
- 4 preformance

prevent untracking of .exe file

prevent of untracking files usfull for focussing git in main project to use this opreator use this code “ *.exe “

0.3 Proses of untracking a tracked file in git

for untracking a file wich tracked before we should to do two steps

- 1 put file in gitignore :
to do this step we use of gitignore code
- 2 delete file in the tracking :
to do this we use this code ‘git rm –cached’

Best practice of git

0.4 Use branch

using branch is usfull to we have a clone of main code so you can expand clone of code in any way and your main code not be changed

branches are usefull for new features , bug fixes , ideas , etc.

0.5 Write good commit

you most write good commit to figure it out who read your code.

you can write about wich lines you add or delete in the code.

0.6 Test your code before commit it

if your code have bug and you commit it the bugs will be saved as your main code.

you can test file before commit to avoide of fix it later.

0.7 Dont commit half down work

you can commit your code when is be a logical part of your code, so devide the code to many logical parts.

if you want have a clean code use 'stash' in git .

0.8 commit related changes

you can commit your code when your code changed a littel .

for example if your code have two bugs you should fix your code in two commits.

that is very esey to read and figure out for who read your code.

Projecr structure

we most use 'gitinit' to creat a git project

use 'git add'and'git commit 'to expent the project

use 'git branch' to creat branches

use 'git checkout' to switch between branches

see above pictures to undrestand:

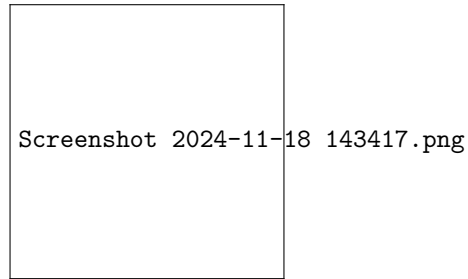


Figure 1: figure of project1 part 1

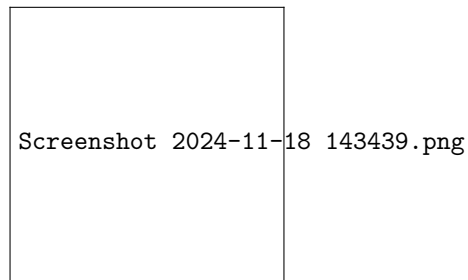


Figure 2: figure of project1 part 2

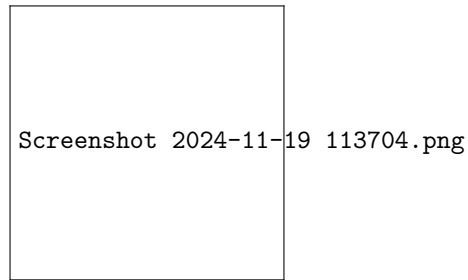


Figure 3: figure of project2 part 1

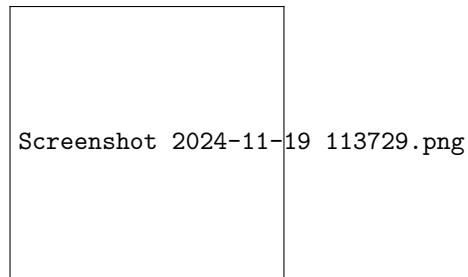


Figure 4: figure of project2 part 2

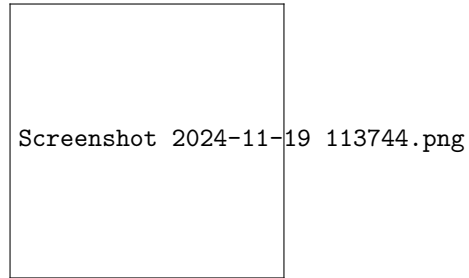


Figure 5: figure of project2 part 3

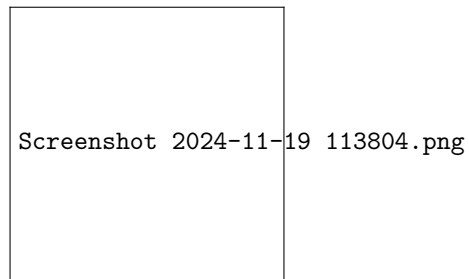


Figure 6: figure of project2 part 4